Homework 7

CS/ECE 374-B Intro to Algorithms & Models of Computation

University of Illinois Urbana-Champaign

Spring 2025

Instructors: Abhishek Umrawal & Ivan Abraham

Maximum Points: 100
Due Date: Wednesday, April 30, 2025, 11:59 PM

Instructions

1. Answer **ALL** questions.
2. Show all the work.
3. Legibly scribe your solutions and upload them as individual problems on Gradescope.

Suggestions

While writing your answers and/or proofs, think about the following.

1. What is it that you are given? What is it that you are assuming? What is it that you are trying to prove?
2. How exactly does a step follow from the previous step? Assume nothing is apparent, even algebraic simplifications. Your reasoning should be clear and convincing.
3. If you are using a result from the class then state it clearly before using it. You should identify where exactly you need that result, cast your problem into the setup of the result, and then use it.
4. If you are using a result from outside the class then you should prove it before using it.

While writing and analyzing an algorithm, note the following.

1. An algorithm is a finite set of primitive and unambiguous instructions to solve a computing problem.
2. Unless otherwise stated, you should give the fastest possible algorithm, i.e., an algorithm with the tightest possible asymptotic time complexity bound.

Problems

1. You are given an undirected graph G with weighted edges and a minimum spanning tree T of G.

   a) Design and analyze an algorithm to update the minimum spanning tree when the weight of a single edge e is decreased.

   b) Design and analyze an algorithm to update the minimum spanning tree when the weight of a single edge e is increased.

   Solution: Denote $G = \{V, E\}$, where V is the set of nodes in G, and E is the set of edges in G. G' is the graph of G with updated weight.

   a) If $e \in T$, then we don't need to do anything, as T remains the MST of G'. If $e \notin T$, then the graph formed by $\{T \cup e\}$ contains exactly one cycle. To maintain $\{T \cup e\}$ a valid MST of G', we will delete the edge with the maximum weight in this cycle. The runtime will be $O(V + E)$.

   b) If $e \notin T$, then we don't need to do anything, as T remains the MST of G'. If $e \in T$, we will delete e from T; after this, there are EXACTLY 2 connected components in T: T_1 and T_2. We traverse every edge in E to find the one with minimum weight, denoted as e', that connects T_1 and T_2. Then the tree formed by $\{T_1 \cup T_2 \cup e\}$ will be the MST of G'. The total runtime is $O(V + E)$.

   One can trivially prove the result is correct, but it's not required by the question; so we omit the tedious work and leave it as an exercise to the reader.

   The brute force will be to re-run Kruskal's algorithm on the updated G' to find the new MST, but that will take $O(E \log V)$ time, which is insufficient.

2. For any integer k, the problem k`SAT` is defined as follows:

   - Input: A Boolean formula Φ in conjunctive normal form, with exactly k distinct literals in each clause.
   - Output: `True` if Φ has a satisfying assignment, and `False` otherwise.

   Design and analyze a polynomial-time reduction from `2SAT` to `3SAT`, and prove the correctness of your reduction.

   Solution: We can easily achieve the reduction by introducing a *dummy variable* Z for each clause.

   Suppose we are given an arbitrary clause $(X_i \vee Y_i)$ in the `2SAT` formula. We convert it into two `3SAT` clauses:

   $$(X_i \vee Y_i \vee Z_i) \quad \text{and} \quad (X_i \vee Y_i \vee \neg Z_i)$$

   where Z_i is a new variable introduced uniquely for this clause.

   We can show that this construction preserves satisfiability in both directions:

   - ($\Rightarrow$) If the original clause $(X_i \vee Y_i)$ is satisfied, then at least one of X_i or Y_i is `True`. Therefore, both $(X_i \vee Y_i \vee Z_i)$ and $(X_i \vee Y_i \vee \neg Z_i)$ are satisfied, regardless of the value assigned to Z_i.
   - ($\Leftarrow$) If both clauses $(X_i \vee Y_i \vee Z_i)$ and $(X_i \vee Y_i \vee \neg Z_i)$ are satisfied, then X_i and Y_i cannot both be `False`. Otherwise, $(X_i \vee Y_i \vee Z_i)$ would force Z_i to be `True`, and $(X_i \vee Y_i \vee \neg Z_i)$ would simultaneously force Z_i to be `False`, leading to a contradiction. Thus, at least one of X_i or Y_i must be `True`, satisfying the original clause.

   Since the construction introduces only a constant number of clauses and variables per original clause, it runs in polynomial time. Therefore, we have a polynomial-time reduction from `2SAT` to `3SAT`.

3. We know that `Clique`(k) is the following NP-hard decision problem: given a graph $G(V, E)$, does G have a clique of size k?

   Consider this secondary decision problem `BigClique`: given a graph $G(V, E)$, does G have a clique of size $\frac{n}{2}$ where $n = |V|$?

   Given a quantum algorithm that solves `BigClique` in constant time. Design and analyze an algorithm that solves `Clique`(k) in polynomial time. Justify the correctness of your algorithm.

   Solution:

Recall that an instance of `Clique` consists of a graph $G = (V, E)$ and integer k. (G, k) is a YES instance if G has a clique of size at least k, otherwise it is a NO instance. For simplicity, we will assume n is an even number.

We describe a polynomial-time reduction from `Clique` to `BigClique`. We consider two cases depending on whether $k \leq n/2$ or not. If $k \leq n/2$ we obtain a graph $G' = (V', E')$ as follows. We add a set of X new vertices where $|X| = n - 2k$; thus $V' = V \cup X$. We make X a clique by adding all possible edges between the vertices of X. In addition, we connect each vertex $v \in X$ to each vertex $u \in V$. In other words $E' = E \cup \{(u, v) \mid u \in V, v \in X\} \cup \{(a, b) \mid a, b \in X\}$. If $k > n/2$ we let $G' = (V', E')$ where $V' = V \cup X$ and $E' = E$, where $|X| = 2k - n$. In other words, we add $2k - n$ new vertices which are isolated and have no edges incident on them.

We make the following relatively easy claims that we leave as exercises.

Claim 1. *Suppose $k \leq n/2$. Then for any clique S in G, $S \cup X$ is a clique in G'. For any clique $S' \in G'$ the set $S' \setminus X$ is a clique in G.*

Claim 2. *Suppose $k > n/2$. Then S is a clique in G' iff $S \cap X = \varnothing$ and S is a clique in G.*

Now we prove the correctness of the reduction. We need to show that G has a clique of size k if and only if G' has a clique of size $n'/2$ where n' is the number of nodes in G'.

$\Longrightarrow$ Suppose G has a clique S of size k. We consider two cases. If $k > n/2$ then $n' = n + 2k - n = 2k$; note that S is a clique in G' as well and hence S is a big clique in G' since $|S| = k \geq n'/2$. If $k \leq n/2$, by the first claim, $S \cup X$ is a clique in G' of size $k + |X| = k + n - 2k = n - k$. Moreover, $n' = n + n - 2k = 2n - 2k$ and hence $S \cup X$ is a big clique in G'. Thus, in both cases, G' has a big clique.

$\Longleftarrow$ Suppose G' has a clique of size at least $n'/2$ in G'. Let it be S'; $|S'| \geq n'/2$. We consider two cases again. If $k \leq n/2$, we have $n' = 2n - 2k$ and $|S'| \geq n - k$. By the first claim, $S = S' \setminus X$ is a clique in G. $|S| \geq |S'| - |X| \geq n - k - (n - 2k) \geq k$. Henc,e G has a clique of size k. If $k > n/2$, by the second claim S' is a clique in G and $|S'| \geq n'/2 = (n + 2k - n)/2 = k$. Therefore, in this case as well G has a clique of size k.

4. Prove that the following problem is NP-Hard: Given an undirected graph G, find *any* integer $k > 374$ such that G has a proper coloring with k colors but G does not have a proper coloring with $k - 374$ colors.

   Solution:

   Let G' be the union of 374 copies of G, with additional edges between *every* vertex of each copy and *every* vertex in *every* other copy. Given G, we can easily build G' in polynomial time by brute force. Let $\chi(G)$ denote the minimum number of colors in any proper coloring of G, and define $\chi(G')$ similarly.

   $\Longrightarrow$ Fix any coloring of G with $\chi(G)$ colors. We can obtain a proper coloring of G' with $374 \cdot \chi(G)$ colors, by using a distinct set of $\chi(G)$ colors in each copy of G. Thus, $\chi(G') \leq 374 \cdot \chi(G)$.

   $\Longleftarrow$ Now fix any coloring of G' with $\chi(G')$ colors. Each copy of G in G' must use its own distinct set of colors, so at least one copy of G uses at most $\lfloor \chi(G')/374 \rfloor$ colors. Thus, $\chi(G) \leq \lfloor \chi(G')/374 \rfloor$.

   These two observations immediately imply that $\chi(G') = 374 \cdot \chi(G)$. It follows that if k is an integer such that $k - 374 < \chi(G') \leq k$, then $\chi(G) = \chi(G')/374 = \lceil k/374 \rceil$. Thus, if we could compute such an integer k in polyomial time, we could compute $\chi(G)$ in polynomial time. But computing $\chi(G)$ is NP-hard!

5. Consider the following search problem:

   `Max-Disjoint-Triples`:

   - Input: a set S of n positive integers and an integer L.
   - Output: pairwise disjoint triples $\{a_1, b_1, c_1\}, \ldots, \{a_{k^*}, b_{k^*}, c_{k^*}\} \subseteq S$, maximizing the number of triples k^*, such that $a_i + b_i + c_i \leq L$ for each i.

   For example, if $S = \{3, 10, 29, 30, 35, 55, 70, 83, 90\}$ and $L = 100$, an optimal solution is $\{3, 10, 83\}, \{29, 30, 35\}$, with two triples (there is no solution with three triples).

   Consider the following decision problem:

`Disjoint-Triples-Decision`:

- Input: a set S of n positive integers, an integer L, and an integer k.
- Output: True iff there exist k pairwise disjoint triples $\{a_1, b_1, c_1\}, \ldots, \{a_k, b_k, c_k\} \subseteq S$, such that $a_i + b_i + c_i \leq L$ for each i.

Prove that `Max-Disjoint-Triples` has a polynomial-time algorithm iff `Disjoint-Triples-Decision` has a polynomial-time algorithm.

Note: One direction should be easy. In `Max-Disjoint-Triples`, the output is not the optimal value k^* but an optimal set of triples, although it may be helpful to give a subroutine to compute the optimal value k^* as a first step.

Solution: It is easy to see that a polynomial-time algorithm for `Max-Disjoint-Triples` implies one for `Disjoint-Triples-Decision`, as we only need to count the number of triples returned by `Max-Disjoint-Triples`, which takes an additional $O(n)$, preserving the polynomial time.

To see that a polynomial-time algorithm for `Disjoint-Triples-Decision` implies one for `Max-Disjoint-Triples`, we may construct the following algorithm *(next page)*.

The validity of the algorithm is very trivial: it brute-forces through all possible triples and finds them one by one. The subroutine `K-Disjoint-Triples` is used to find the $k*$, and has a running time of $O(n\times$ `Disjoint-Triples-Decision`). The implemented `Max-Disjoint-Triples` first finds the optimal k, and then uses four nested for-loops to find the triples, which gives a multiplier of $O(n^4)$ to the running time of `K-Disjoint-Triples`. Thus, the total running time would be $O(n^4 \times n\times$ `Disjoint-Triples-Decision`), which would still be polynomial if `Disjoint-Triples-Decision` has polynomial time. □

```
 K-Disjoint-Triples(S, L):                                    # subroutine to find k*
for k ← 1 to ⌊n/3⌋ + 1:
  if  Disjoint-Triples-Decision(S, L, k) is False:
    return k − 1

Max-Disjoint-Triples(S, L):
K ←  K-Disjoint-Triples(S, L)
Triples ← Array[1, ..., K; 1, 2, 3]
for k ← K to 1:
  found ← False
  n ← length(S)
  for i ← 1 to n − 2:                                  # search by index i < j < m
    for j ← i to n − 1:
      for m ← j to n:
        if  K-Disjoint-Triples(S - S[i] - S[j] - S[m], L) = k − 1 :
          Triples[k] ← [i, j, m]
          found ← True
          S ← S - S[i] - S[j] - S[m]                       # exclude the found triples
      if found = True:
        break
    if found = True:
      break
return Triples
```